SUNWAY UNIVERSITY

Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

Topic 02:
System Models

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

NET3204
Distributed Systems

Chee Sun LIEW cheesunl@sunway.edu.my

Learning Outcome

- At the end of this lecture, you will be able to:
  - Understand the concept of system modelling.
  - Explain the types of conceptual models with examples.
  - Describe the design requirements of distributed system architecture.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

System models

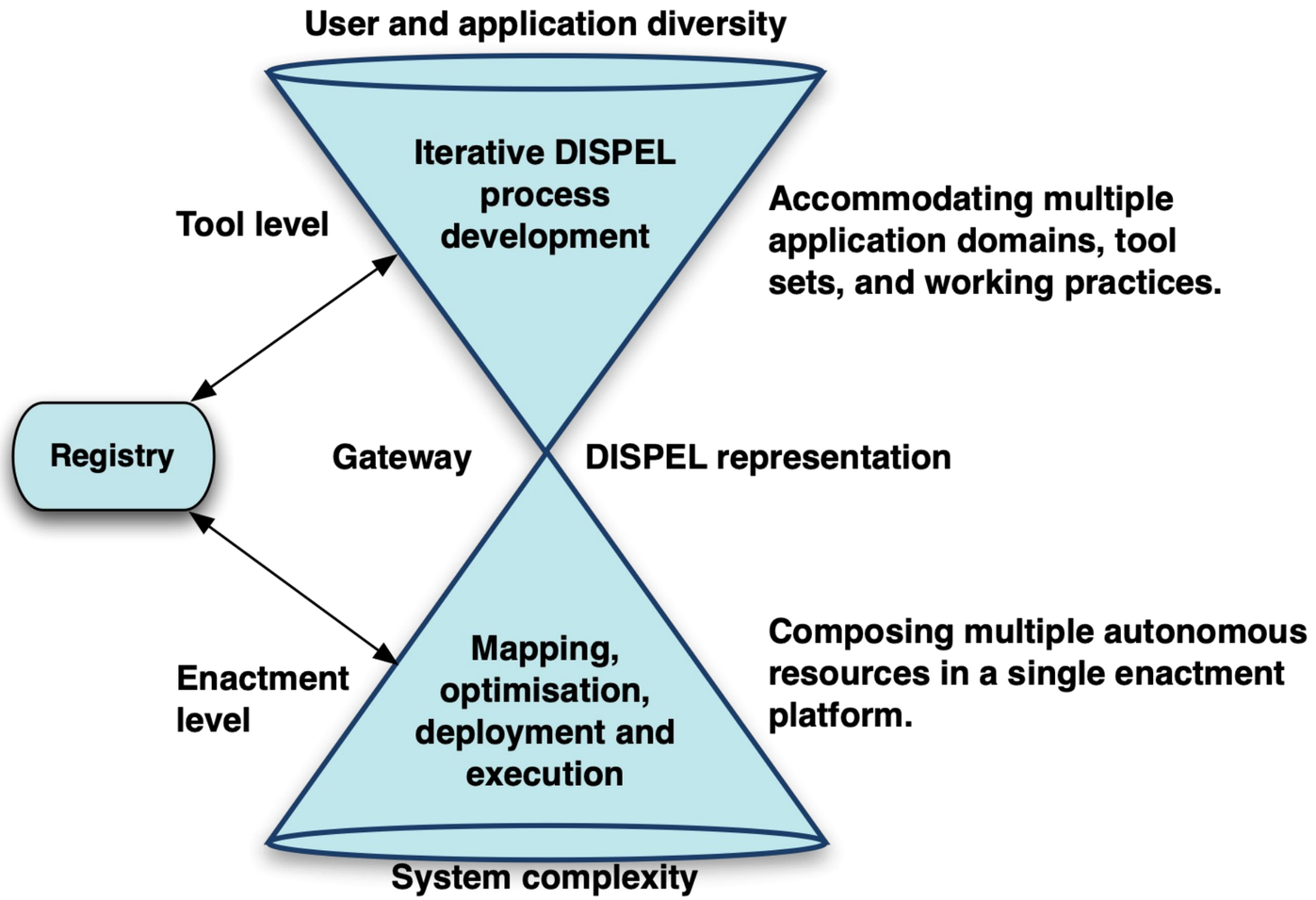
- Conceptual model intended to provide an abstract, simplified but consistent description of a relevant aspect of distributed system design.
- Types of conceptual models:
  - Architectural model
  - Fundamental model

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Architectural model

- Provides a high level view of the distribution of functionality between components and the relationships (communication pattern) between them.
- Determines the distribution of data and the computational tasks amongst the physical nodes of the system.
- Helpful when evaluating properties such as performance, reliability, scalability.

Example: DISPEL architecture



Advanced Data Mining and Integration Research for Europe
http://www.admire-project.eu/

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

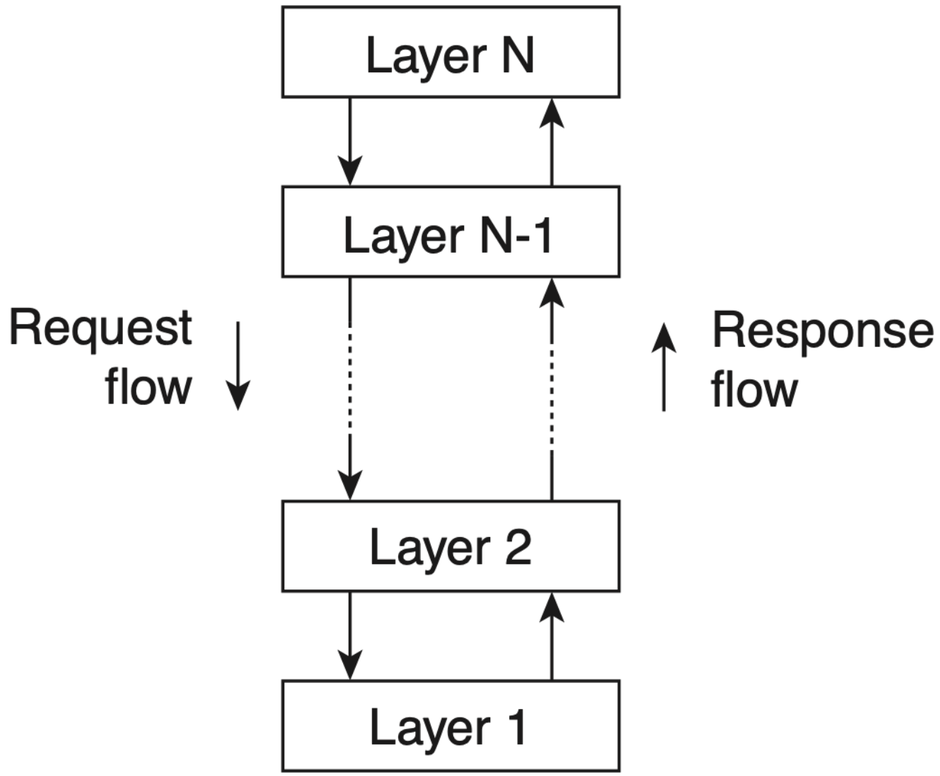
Architectural model



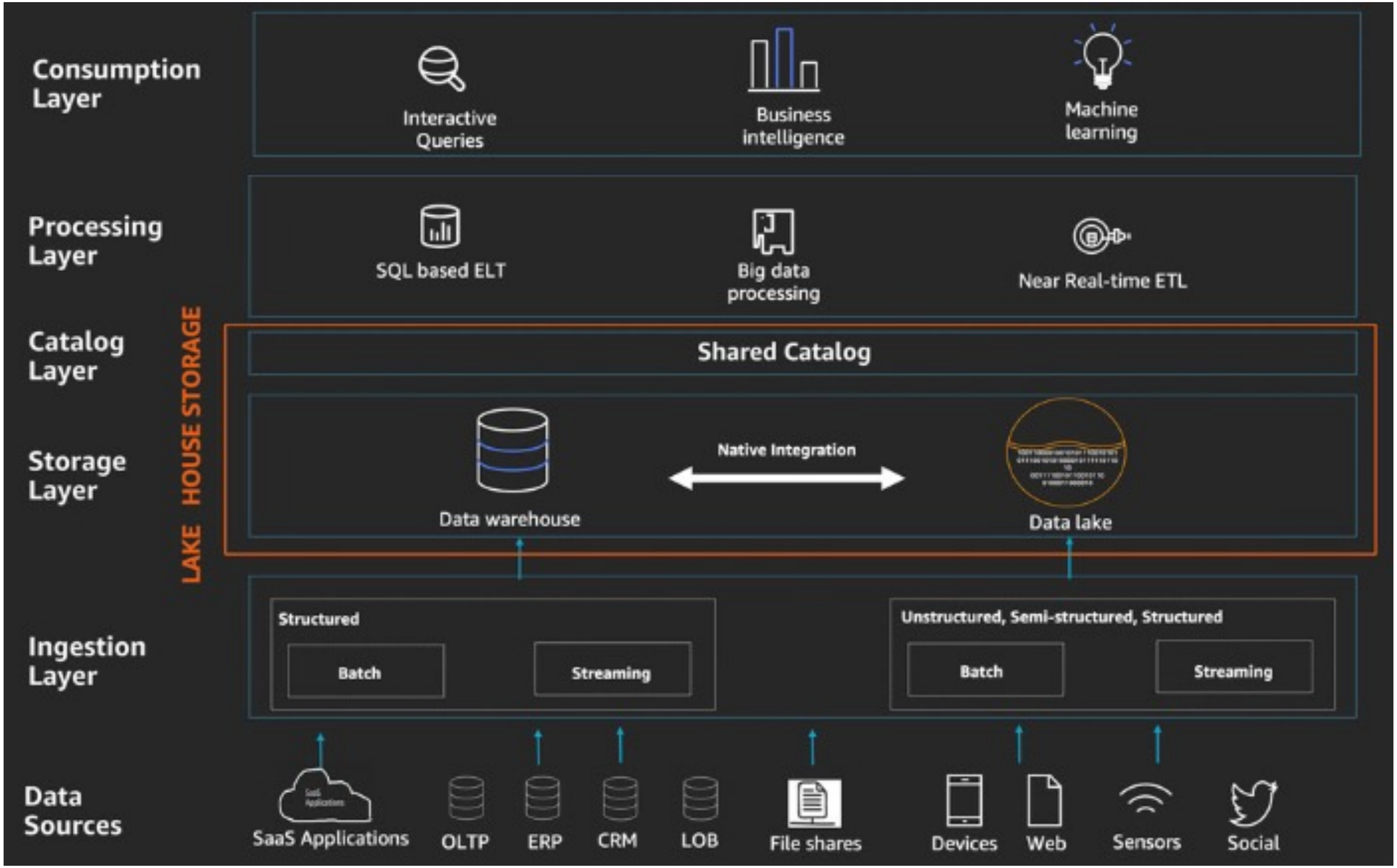
- Analogy:
  - Architectural design of a building defines:
    - how it looks like
    - structure
    - define style
    - provide consistent frame of reference for the design.
- Architectural model goal:
  - ensure that the structure will meet present and likely future demands on it,
  - make the system reliable, manageable, adaptable and cost-effective.

Layered architecture

- Used for client-server system
- Variations:
  - Single-tier
    - terminal-mainframe
  - Two-tier
    - client-server
  - Three-tier
    - UI-processing-data



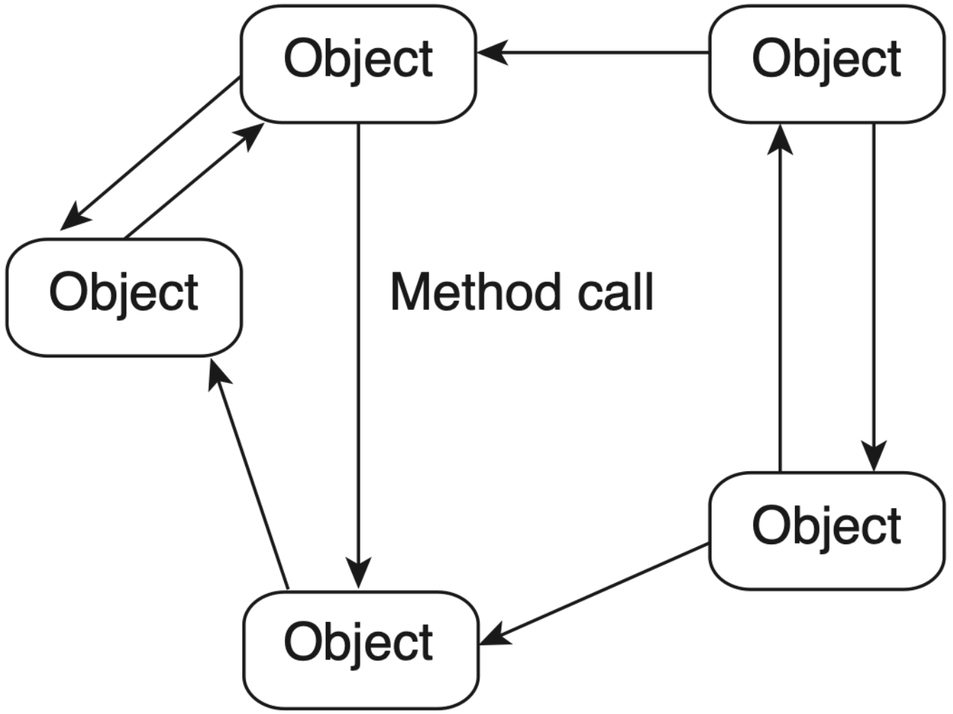
OUR VISION: TO BE A WORLD CLASS UNIVERSITY



What is Architecture Diagramming? https://aws.amazon.com/what-is/architecture-diagramming/

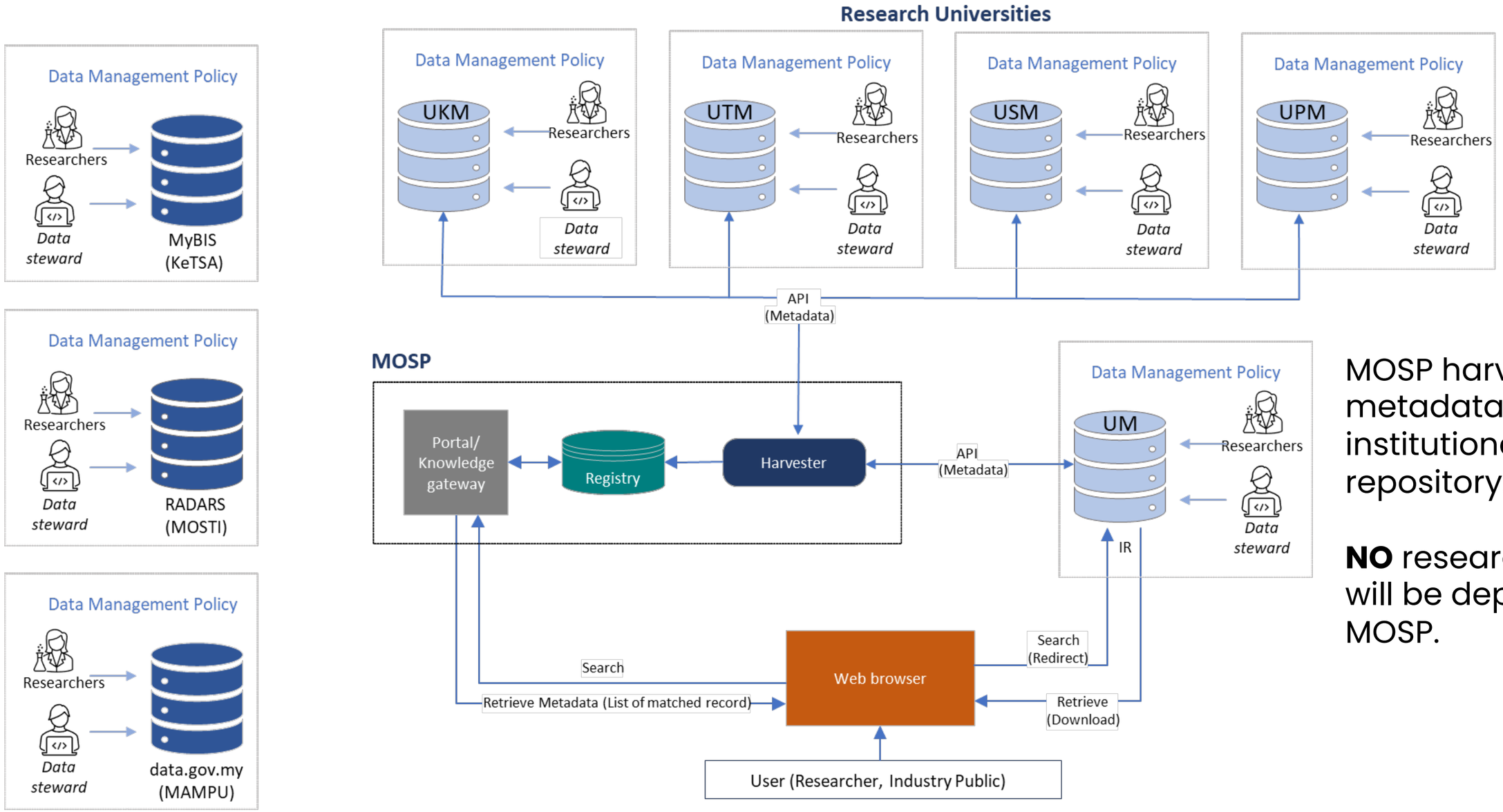
Object-based architecture

- Used in distributed-object systems
  - extends object-oriented programming to distributed systems, that allows objects to be distributed across multiple nodes.
  - e.g. Java RMI, CORBA



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

MOSP Architecture

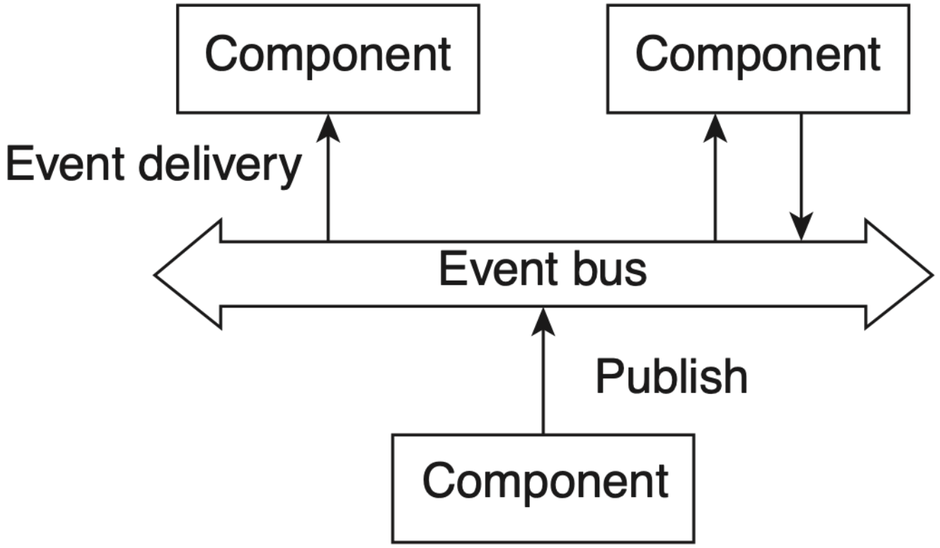


MOSP harvest the metadata from the institutional repository.

NO research data will be deposited in MOSP.

Event-based architecture

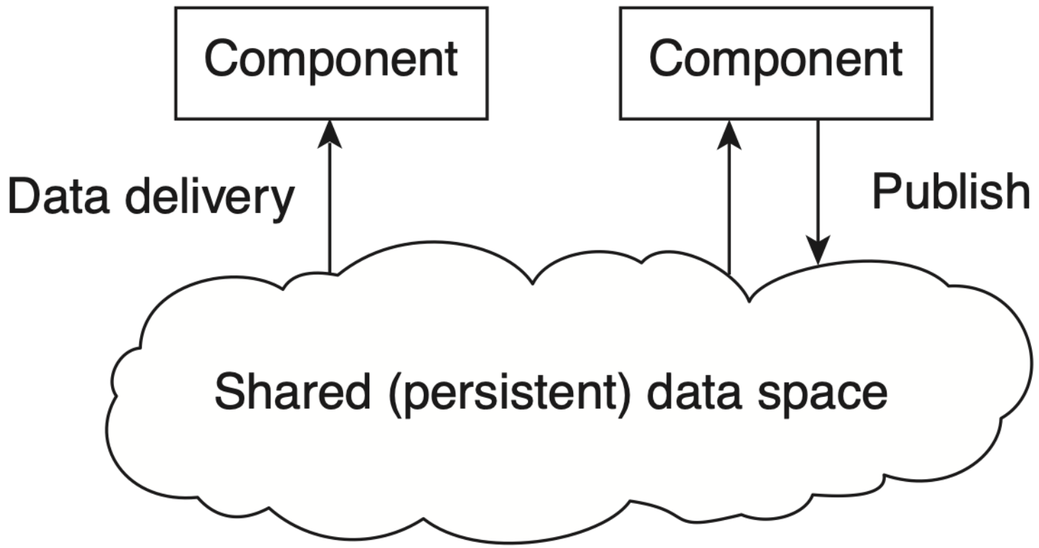
- Used in event-driven systems, e.g. sensor network, traffic congestion control trading systems, whose execution is response to the arrival of events
- asynchronous message-passing rather than request-reply



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Shared data-space architecture

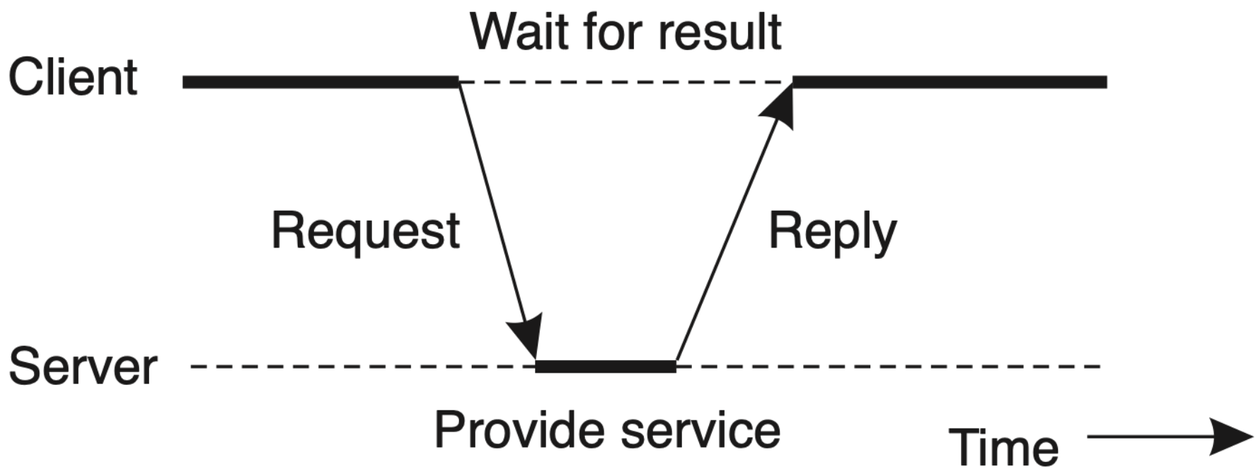
- Processes are decoupled in space and time
  - data publishers post items to shared space; data consumers pick up at a later time



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Centralised Architectures

- Client-server Model:
  - There are processes offering services (servers)
  - There are processes that use services (clients)
  - Clients and servers can be on different machines
  - Clients follow request/reply model to using services



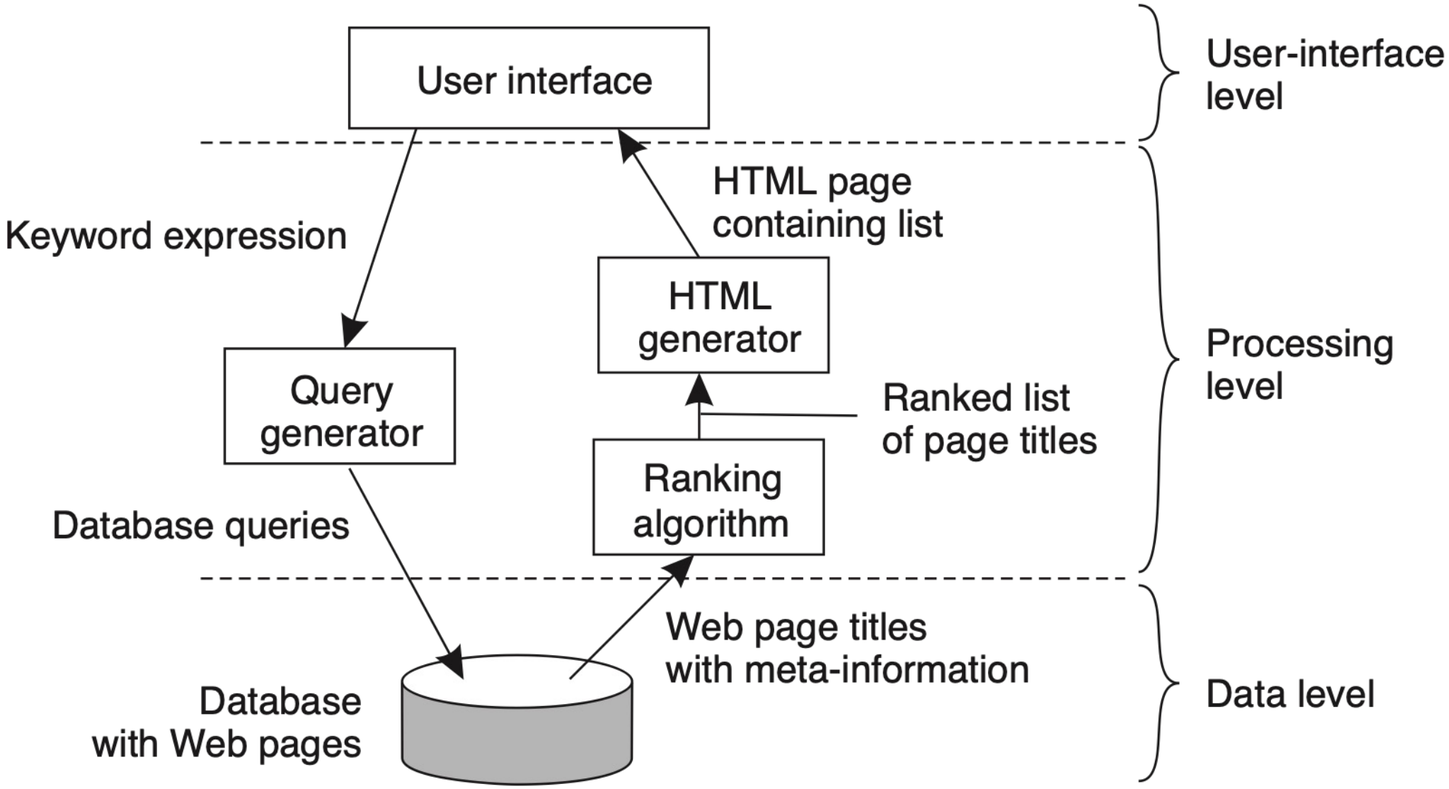
OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Application layering

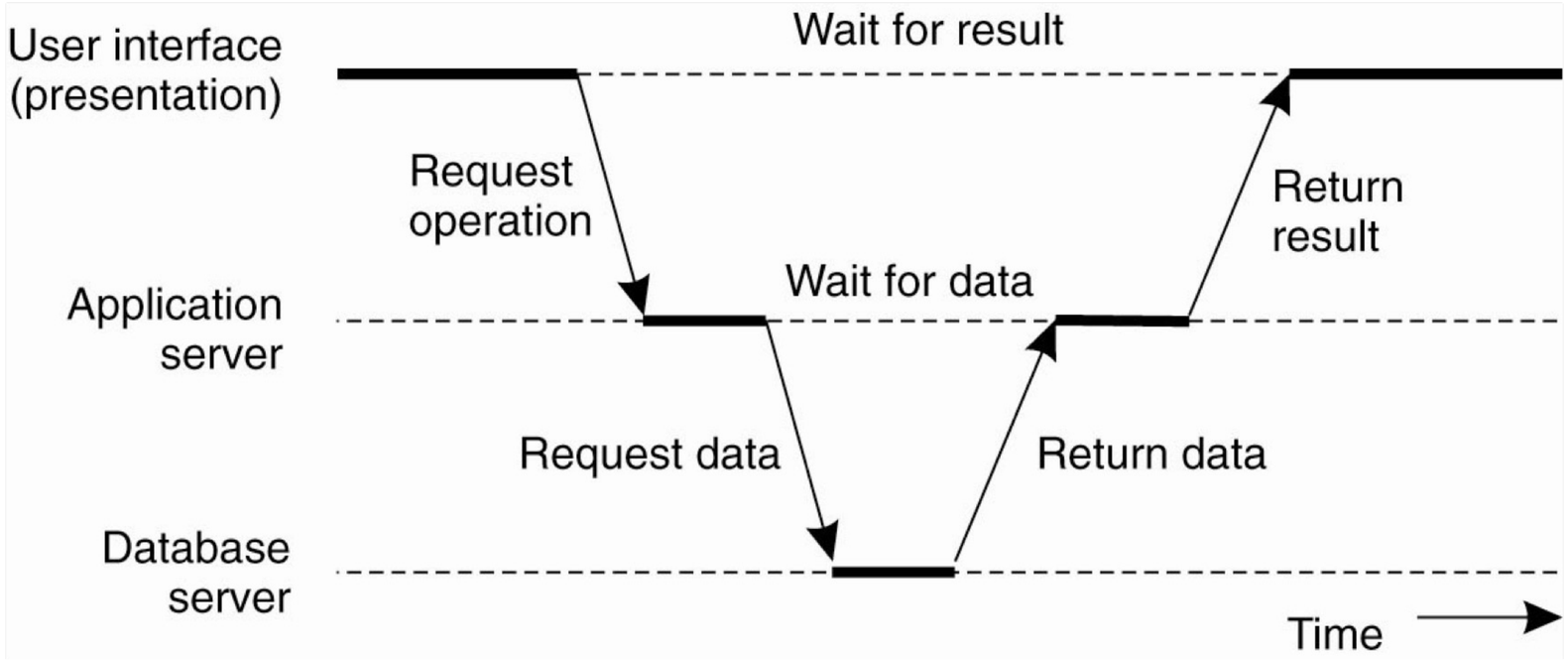
- *User-interface layer* contains units for an application's user interface
- *Processing layer* contains the functions of an application, i.e. without specific data
- *Data layer* contains the data that a client wants to manipulate through the application components

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Application layering

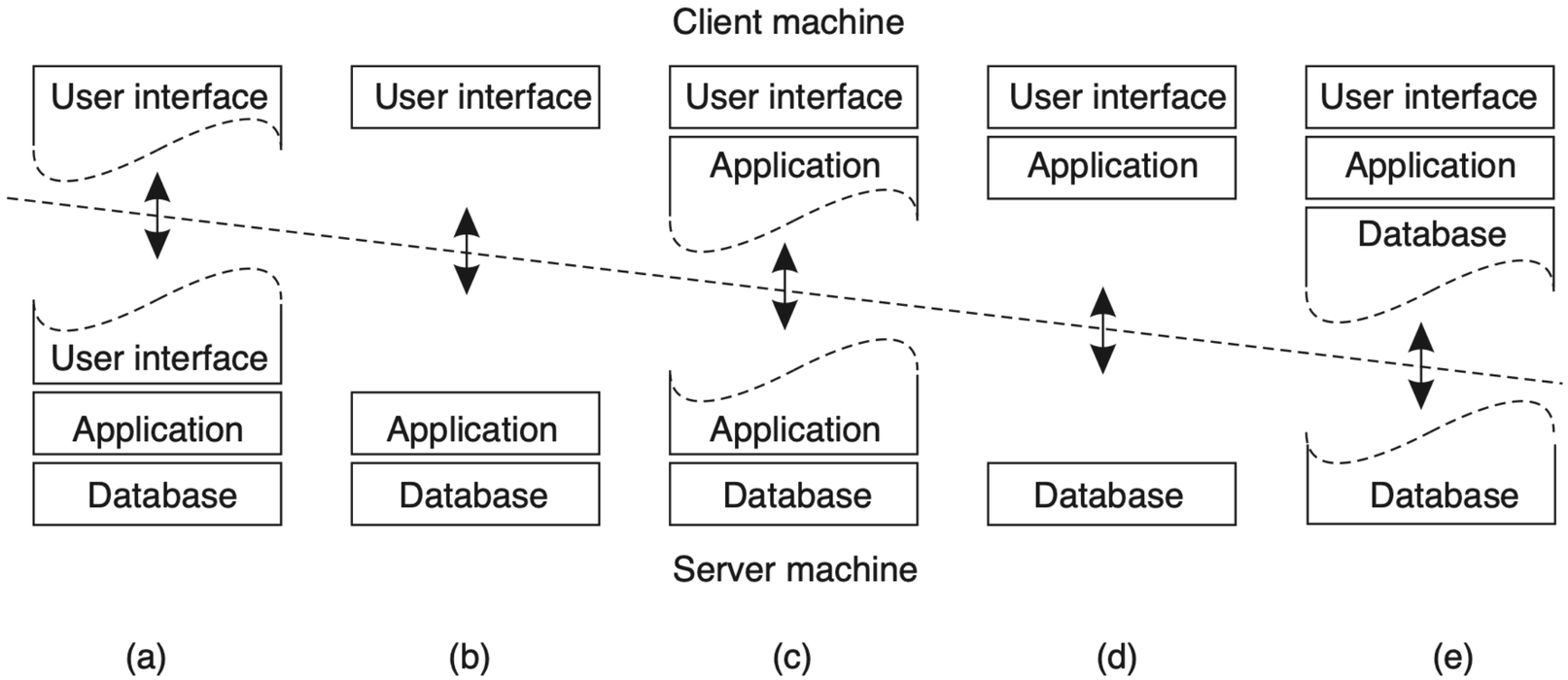


Multi-tiered architectures



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Multi-tiered architectures



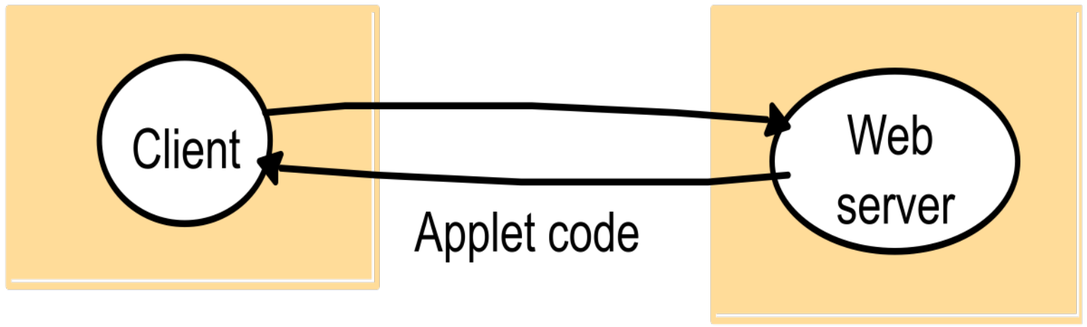
Variations on client-server model

- Variations on the client-server model can be derived from the consideration of the following factors:
  - the use of mobile code and mobile agents
  - users' need for low-cost computers with limited hardware resources that are simple to manage
  - the requirement to add and remove mobile devices in a convenient manner

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Mobile code

a) client request results in the downloading of applet code



b) client interacts with the applet



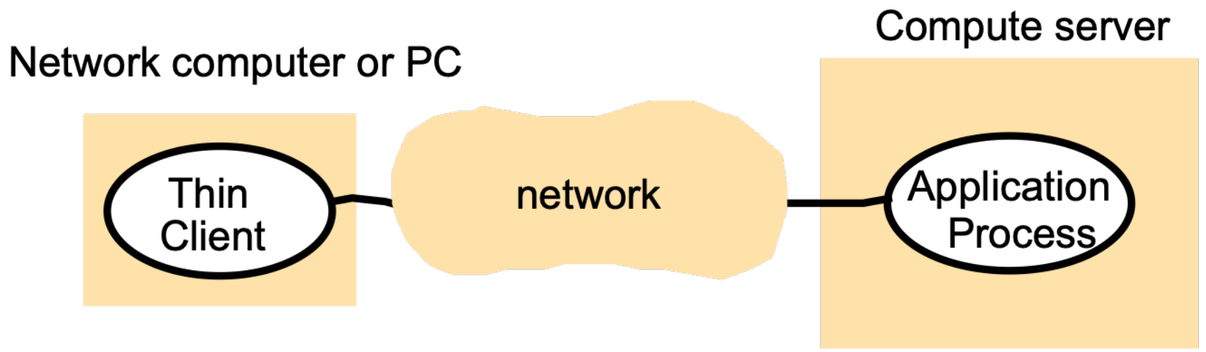
Mobile agent

- A mobile agent is a running program (both data and code) that travels from one computer to another in a network carrying out a task on someone's behalf.
- Compared to static agents that make remote invocations to some resources, there is reduction in communication costs and time.
- Example :
  - install and maintain s/w on the computers within an organization
  - compare the prices of products.

- How about security?
- Problem – when the mobile agents are refused to access information.

Network computers & thin clients

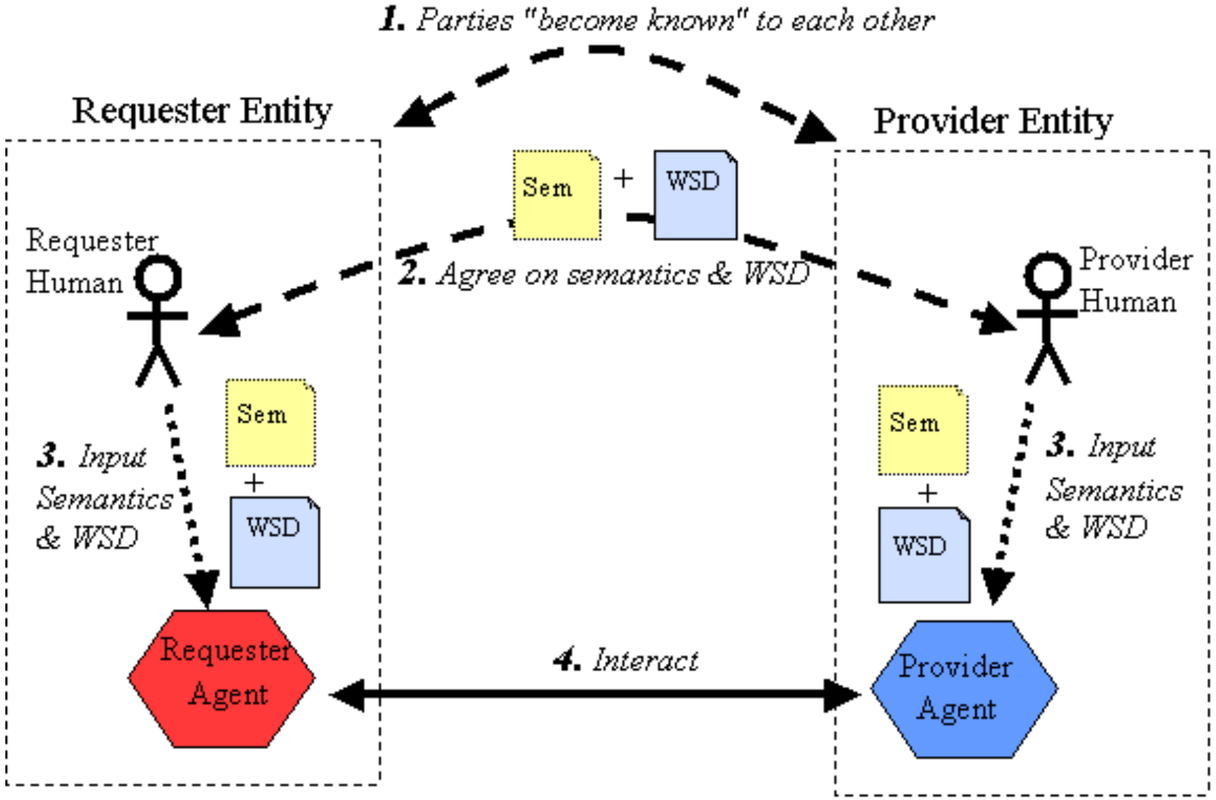
- Network computers – downloads its own OS and application s/w needed by the user from a remote file server. Applications are run locally but the files are managed by a remote file server.



- Thin clients – refers to a software layer that supports a window-based user interface on a computer that is local to the user while executing application programs on a remote computer.
  - Problem – delays in highly interactive graphical activities.

Web services

- A Web service is a software system designed to support interoperable machine-to-machine interaction over a network



Web Services Architecture, from W3C Working Group
http://www.w3.org/TR/ws-arch

Web services

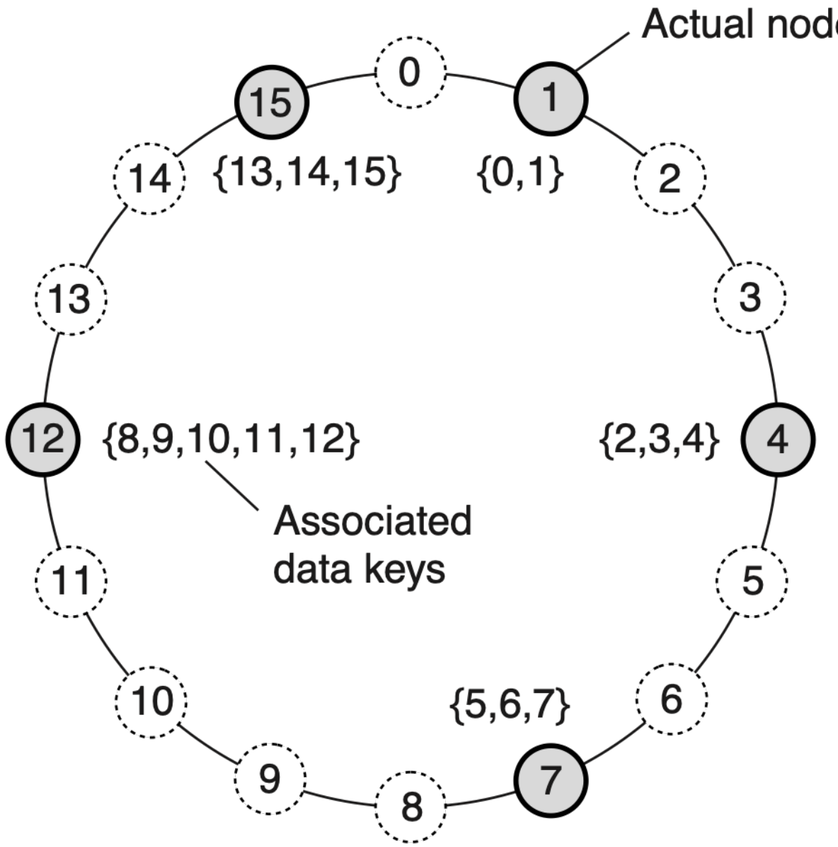
- The Web Service Description (WSD)
  - a machine-processable specification of the Web service's interface, written in WSDL.
  - defines the message formats, datatypes, transport protocols, and transport serialisation formats that should be used between the requester agent and the provider agent.
- The semantics of a Web service
  - the shared expectation about the behaviour of the service, in particular in response to messages that are sent to it.
  - the "contract" between the requester entity and the provider entity regarding the purpose and consequences of the interaction.

Decentralised Architectures

- Peer-to-peer systems
  - Structured P2P
    - nodes are organized following a specific distributed data structure
  - Unstructured P2P
    - nodes have randomly selected neighbors
  - Hybrid P2P
    - some nodes are appointed special functions in a well-organised fashion

Structured P2P

- Organise the nodes in a structured *overlay network* such as a logical ring, and make specific nodes responsible for services based only on their ID.
- The system provides an operation LOOKUP(key) that will efficiently *route* the lookup request to the associated node.



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

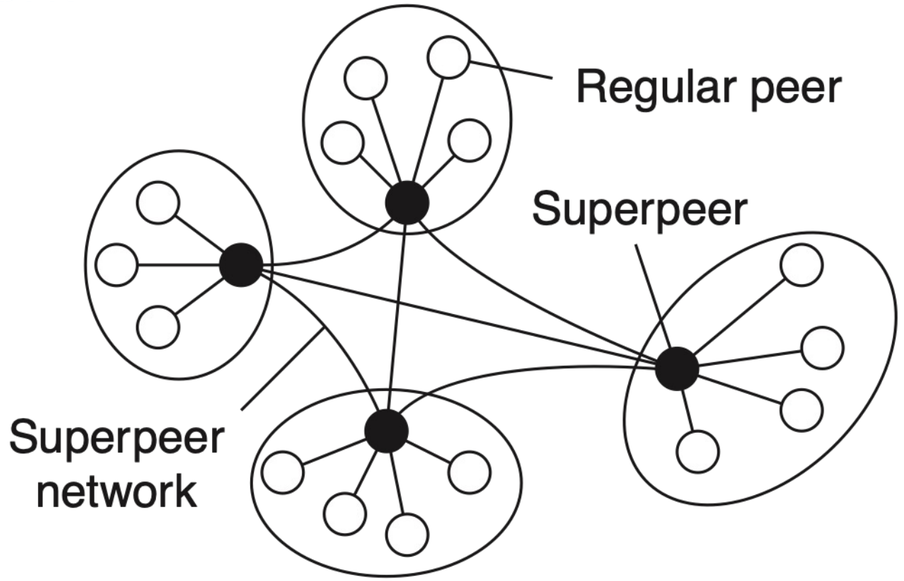
Unstructured P2P

- Many unstructured P2P systems attempt to maintain a random graph.
- Each node is required to contact a randomly selected other node:
  - Let each peer maintain a partial view of the network, consisting of c other nodes
  - Each node P periodically selects a node Q from its partial view
  - P and Q exchange information and exchange members from their respective partial views

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Superpeers

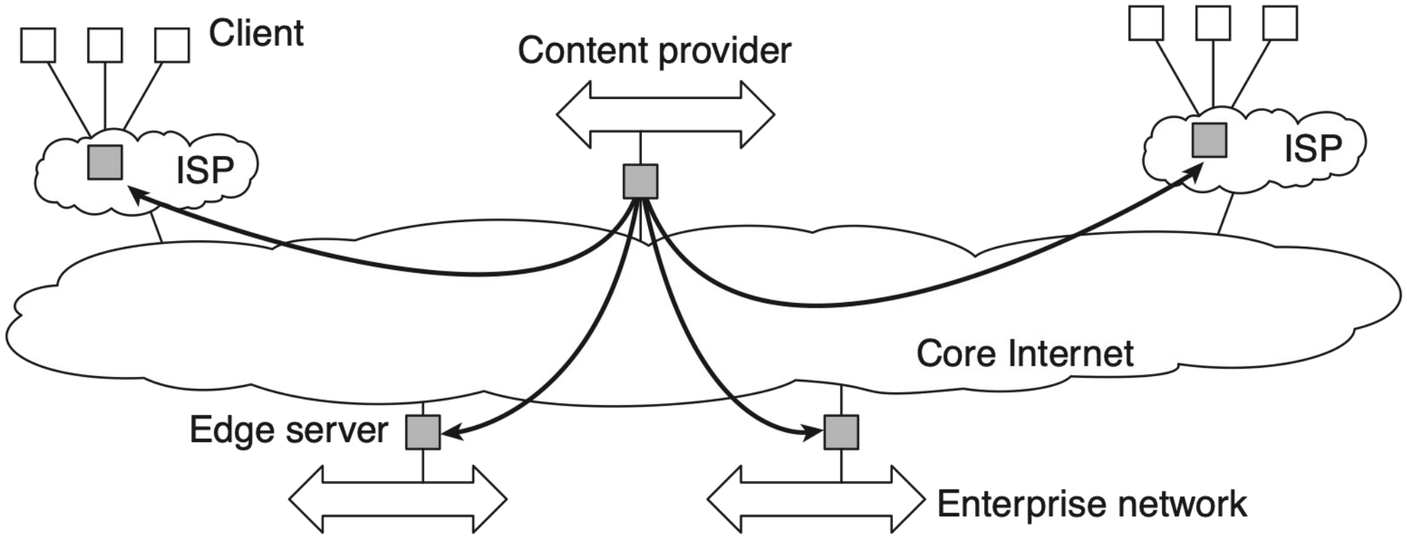
- Sometimes it helps to select a few nodes to do specific work, i.e. superpeer.
- Example:
  - Peers maintaining an index (for search)
  - Peers monitoring the state of the network
  - Peers being able to setup connections



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Hybrid Architectures

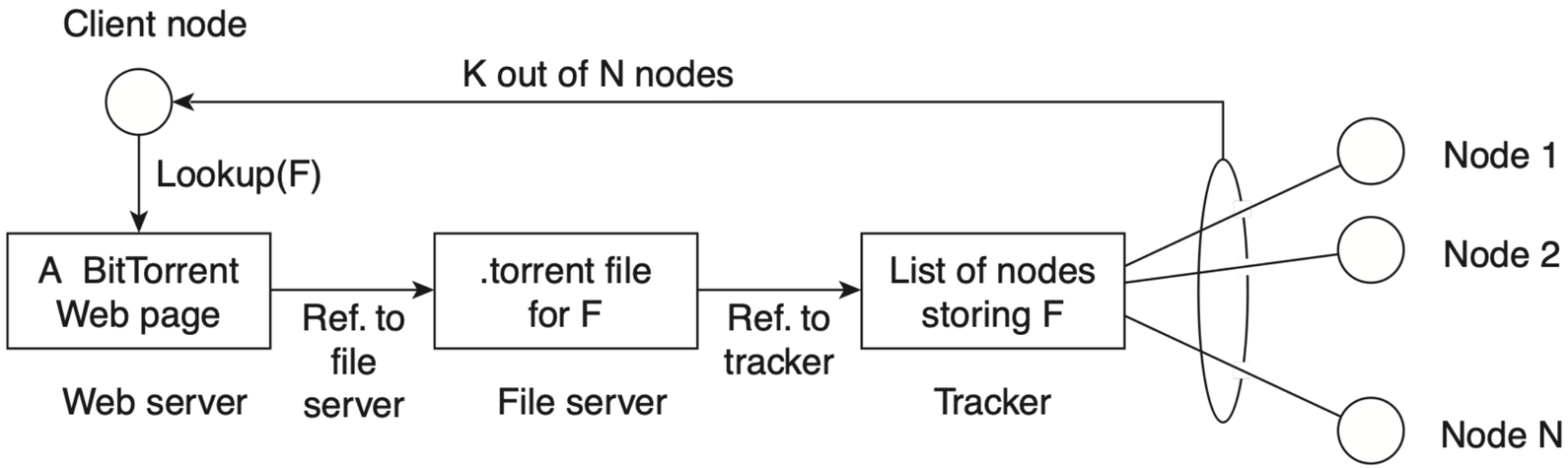
- Client-server combined with P2P



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Hybrid Architectures - BitTorrent

- Once a node has identified where to download a file from, it joins a swarm of downloaders who in parallel get file chunks from the source, but also distribute these chunks amongst each other.



Design requirements for distributed architectures

- Performance issues – arising from the limited processing and communication capacities of computers and networks.
  - Responsiveness: need fast and consistent response.
    Identify the delay: -
    - load & server performance
    - Network
    - s/w components (OS and middleware services in client and server)
  - Throughput:

- Quality of Service (QoS)
  - Reliability, security, performance, and adaptability.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Design requirements for distributed architectures

- Use of caching and replication
  - Web-caching protocol
  - Client request – reply by cached by the browser, or by proxy server.

- How to know if it is up-to-date?
  - Possibilities:
    - Up to date
    - Not up to date
      - Inform browser and proxy server
      - Browser and proxy server checked with web server
      - Expiry time by web server

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Design requirements for distributed architectures

- Dependability issues: Defined as correctness, security and fault tolerance.
  - Correctness: checking or ensuring the correctness of DS and concurrent program.
  - Security: Locate, and protect sensitive data. e.g. patient's record.
  - Fault tolerance: Dependable application should continue to function correctly in the presence of faults (in s/w, h/w or networks).

- One way to solve the problem is Redundancy:
  - multiple computers
  - multiple communication paths
  - replicas of data

Fundamentals Models

- Provides a vertical view representing some key aspects of distributed systems.
- Messages communication over a computer network can be affected by:
  - delays
  - can suffer from a variety of failures
  - vulnerable to security attacks
- These issues are addressed by 3 models:
  - Interaction model – define performance and setting time limit for message delivery.
  - Failure model – specification of faults. It defines reliable communication and correct processes.
  - Security model – discuss possible threats.

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Fundamentals Models – Interaction Model

- Concerned with the performance of processes and communication channels and the absence of a global clock.
- The processes interact by passing messages (i.e. information flow) and coordination between processes (i.e. synchronisation and ordering of activities).
- Consideration: delays; accuracy affected by these delays; no single time.
- Performance of communication channels: Involves sending of streams/message passing. Performance characteristics:
  - Latency : the delay between sending of a message and its receipt.
  - Bandwidth : total amount of information that can be transmitted over it in a given time.
  - Jitter : the variation in the time taken to deliver a series of messages. Relevant to multimedia data

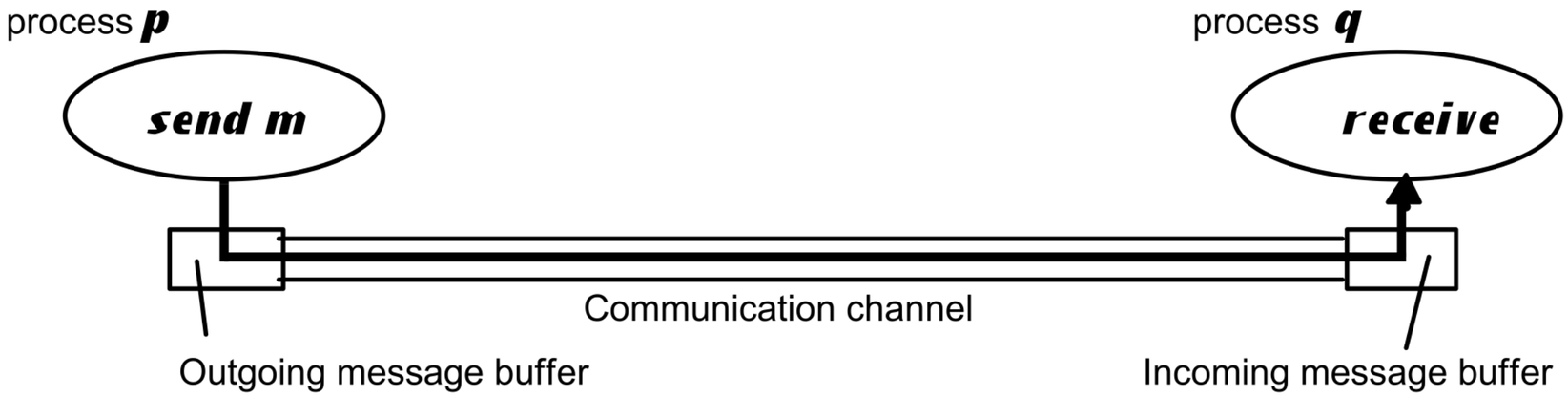
Fundamentals Models – Interaction Model

- Computer clocks and timing events:
  - Problem – clock drift.
  - One solution – GPS
  - Each computer has internal/local clock.

- Two variants of the interaction model:
  - Synchronous DS (strong assumption of time)
    - Know process execution time, transmission time, and clock drift rate.
  - Asynchronous DS (no assumptions about time)
    - Arbitrary process execution time, message transmission time, and clock drift.

Fundamentals Models – Failure Model

- Classifies the failure of processes and basic communication channels in a distributed system.
- The failure model defines the ways in which failure may occur in order to provide an understanding of the effects of failures.

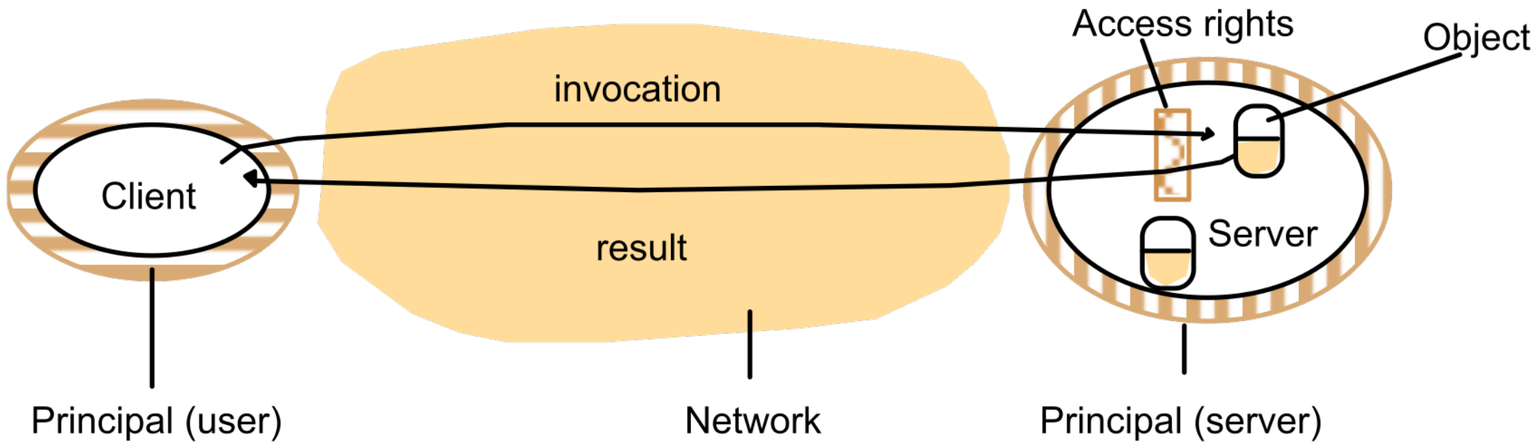


Type of failures

- Omission failures – a process or communication channel fails to perform actions that it is supposed to do.
  - Channel omission (affects channel)
    - A message inserted in an outgoing message buffer never arrives at the other end's incoming message buffer.
  - Send-omission (affects process)
    - A process completes a *send*, but the message is not put in its outgoing message buffer.
  - Receive-omission (affects process)
    - A message is put in a process's incoming message buffer, but that process does not receive it.
- Arbitrary failures – any type of error may occur. e.g. a process sets wrong values in its data items; a process returns a wrong value to a request.
- Timing failures – Applicable to synchronous DS only where time limits are set on processes execution time, message delivery time and clock drift rate.

Fundamentals Models – Security Model

- Identifies the possible threats to processes and communication channels in an open distributed system.
- The security of a DS can be achieved by securing the processes and the channels used for their interactions and by protecting the objects that they encapsulate against unauthorised access.
- Protecting objects – give access right.



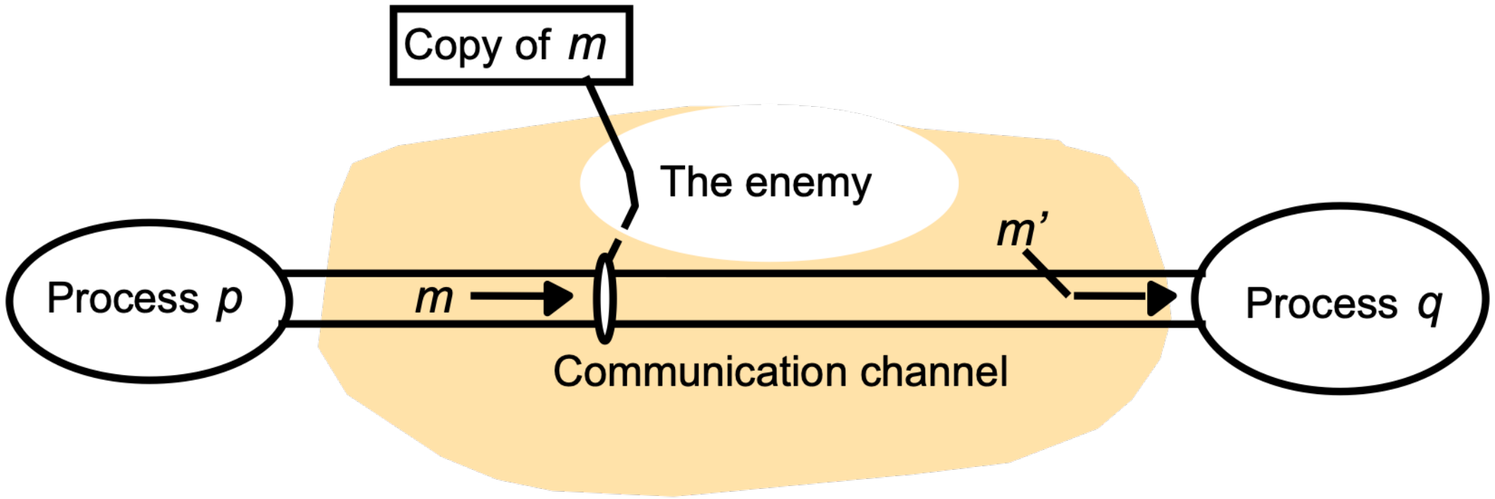
OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Fundamentals Models – Security Model

- Securing processes and the channels used for interactions
  - Processes interact by sending messages and these messages are exposed to attack esp. financial transactions, and confidential and classified information.
  - In order to identify and defeat them, we have to analyze these threats by having a security model.

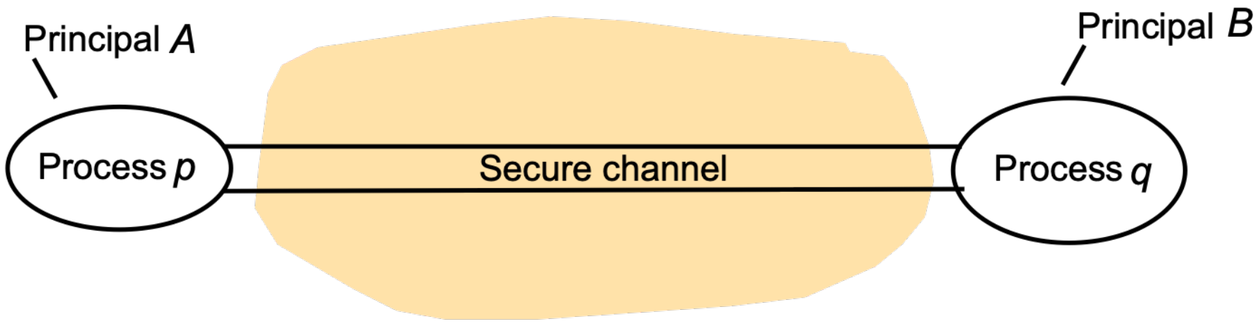
Fundamentals Models – Security Model

- Threats from potential enemy
  - Threats to processes
  - Threats to communication channels
  - Denial of service



Fundamentals Models – Security Model

- Defeating Security Threats:
  - Several measures can be implemented:
    - Shared secrets - Both sender and recipients know a secret.
    - Cryptography - is the science of keeping messages secure, and encryption is the process of scrambling a message in such a way as to hide its contents.
    - Authentication – proving the identities supplied by their sender.



Summary

- What did I learn in this module?
  - The concept of system model
  - The types of conceptual model
  - The distinguishable features among all of the architectural models
  - The design requirements of distributed system architecture
  - The three fundamental of distributed systems

OUR VISION: TO BE A WORLD CLASS UNIVERSITY

THANK YOU

Chee Sun LIEW
cheesunl@sunway.edu.my

